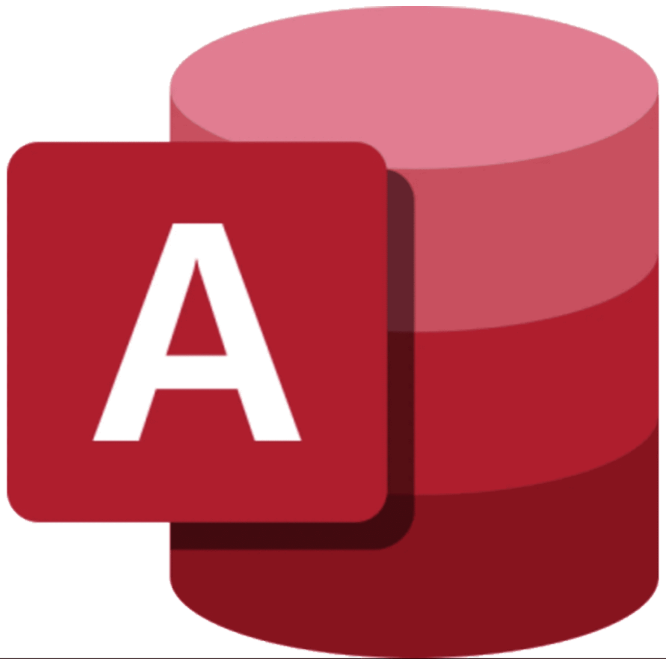
Raphaël Hennocq
Youssef Khemira
Simon Trichard

NF22
Projet Access
Gestion de covoiturage



Sommaire

I/ Rappel du cahier des charges et utilisateur type 3
II/ Mode d'emploi 4
1) Se connecter 4
2) Choisir le menu 5
a) Menu conducteur 5
b) Menu passager 5
3) Effectuer une action 6
III/ Modèle conceptuel et Modèle logique 7
1) Modèle conceptuel 7
2) Modèle logique 7
IV/ Fonctionnement technique 8
1) Requêtes 8
2) Formulaires 9
V/ Retour bêta-testeur 11
V/ Discussion 11
a) Contributions 11
i) Partie de Raphaël seul 11
ii) Partie de Simon et Raphaël 12
iii) Partie de Youssef 12
b) Voies d'améliorations 13

I/ Rappel du cahier des charges et utilisateur type

Une société de covoiturage qui souhaite se servir du logiciel Access afin de mettre des personnes possédant des voitures et des personnes sans voiture. La première chose à mettre en place est un système d'inscription pour enregistrer et attribuer des informations à chaque utilisateur de l'application. Chaque utilisateur devra avoir son mot de passe pour que ses données lui soient propres. En s'inspirant des grandes plateformes de covoiturages actuelles, chaque utilisateur devra pouvoir naviguer sur une interface qui lui est destinée en fonction de son rôle lors du potentiel covoiturage. Deux formulaires différents devront donc être développés, un pour les passagers et un pour les conducteurs :

- Pour les passagers, il faudra mettre en place des formulaires afin qu'ils soient en capacité de consulter les fiches existantes ou faire une nouvelle demande.
- Pour les conducteurs, il faudra mettre en place des formulaires afin qu'ils soient en capacité d'accéder aux demandes ou proposer un nouveau trajet.

Lorsqu'une demande est acceptée par un conducteur, le passager ayant fait la demande de trajet est automatiquement assigné à ce trajet en tant que passager. Il peut aussi suivre l'avancement de ses réservations et demandes de trajet via un menu dédié.

Les données qui devront être enregistrées pour le bon fonctionnement de l'interface sont :

- Pour le conducteur : e-mail, nom, prénom, titre (madame, monsieur), adresse, code postal, ville, téléphone, voiture (marque, modèle, couleur, numéro d'immatriculation).
- Pour le passager : e-mail, nom, prénom, titre (madame, monsieur), adresse, code postal, ville, téléphone.
- Pour le trajet : Lieu de départ, lieu d'arrivée, Jours ou dates, heure d'arrivée, période, impact carbone.

Un utilisateur peut être conducteur sur un trajet et passager sur un autre sans problème. La différence entre les conducteurs et passagers est que si un utilisateur souhaite proposer un trajet, il doit avoir ajouté une voiture à sa liste de véhicules. Ainsi, le fait d'avoir une voiture ajoutée ou non ne représente pas la différence entre un conducteur et un passager.

L'utilisateur type de la plateforme possède deux profils différents :

- Passager : Il souhaite se déplacer mais n'est pas véhiculé et souhaite économiser de l'argent donc le covoiturage est intéressant pour lui. Il est potentiellement écologique et voit dans le covoiturage un moyen de réduire

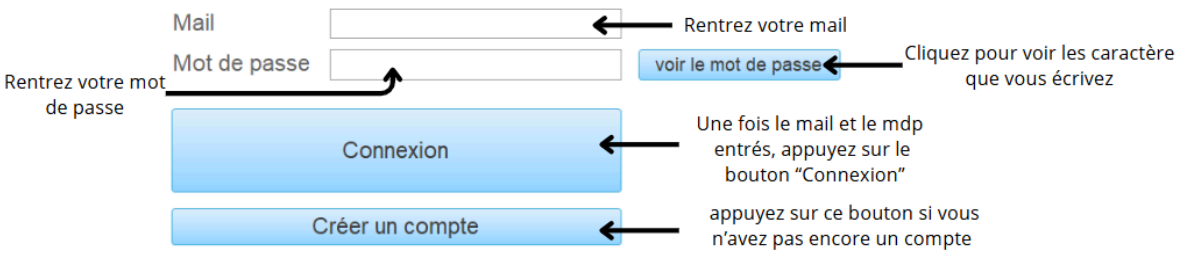
son impact carbone. Il a des attentes sur les horaires du trajet qu'il veut prendre et n'accepte pas les trajets qui ne lui sont pas profitables.

- Conducteur : Il possède une voiture et effectue des trajets avec. Il souhaite réduire ses dépenses, notamment sur l'essence et les péages et voit dans le covoiturage un moyen efficace de diminuer ses coûts. Comme les passagers, le conducteur peut aussi être sensible à son empreinte carbone et considère le covoiturage comme une bonne action pour la planète.

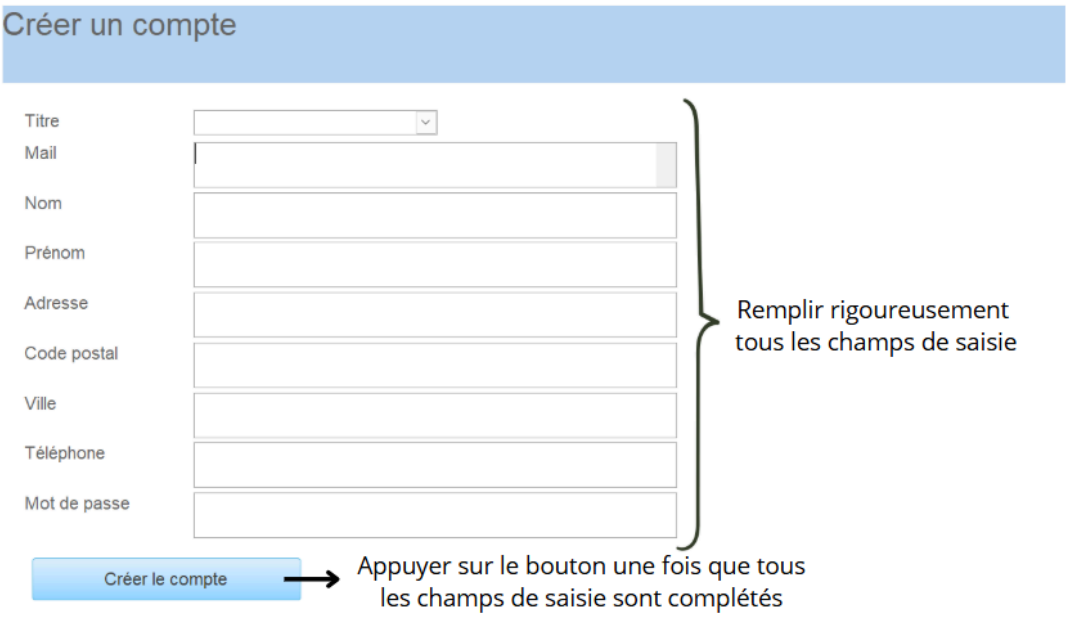
II/ Mode d'emploi

1) Se connecter

Lorsque l'utilisateur lance l'application il doit se connecter en entrant son mail est son mot de passe avant d'appuyer sur le bouton connexion.

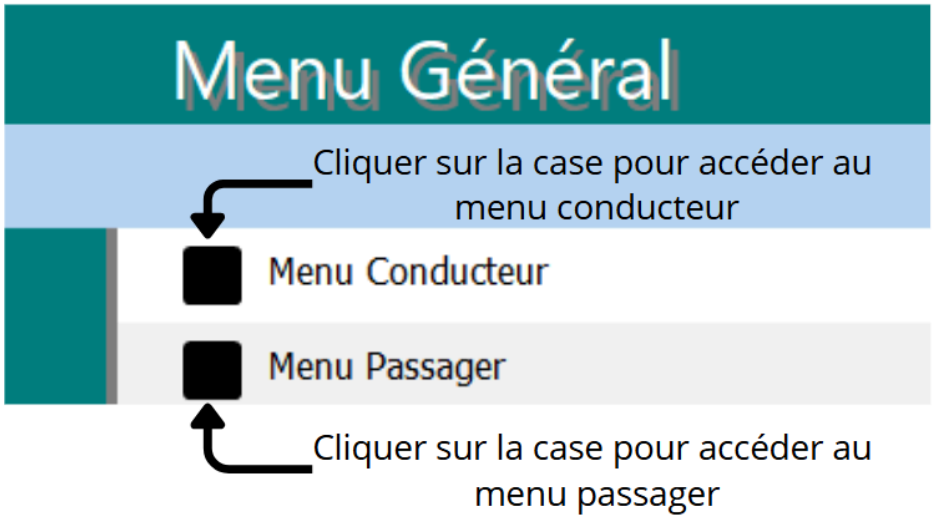


Il est aussi possible de s'inscrire à l'aide d'un bouton situé sous le bouton de connexion. L'utilisateur est alors redirigé vers un formulaire de création d'utilisateur :



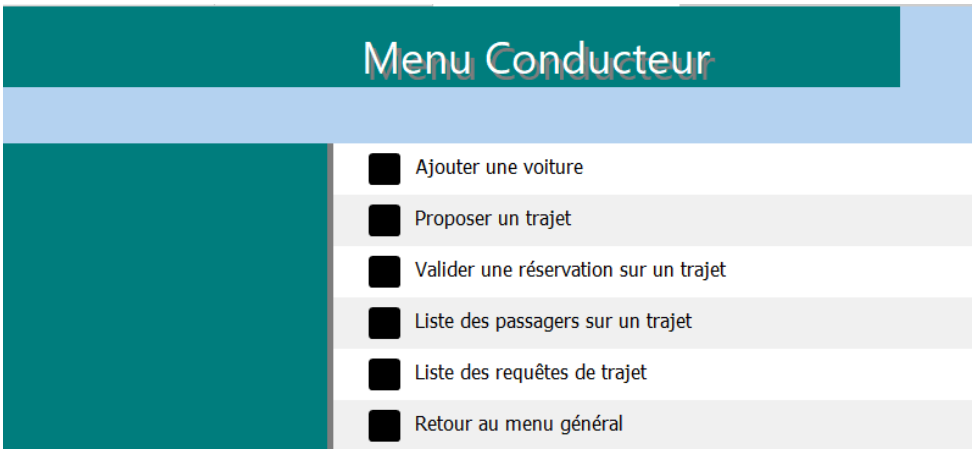
2) Choisir le menu

Une fois connecté, l'utilisateur doit choisir entre deux menus : soit le menu destiné au conducteur, soit le menu destiné au passager. Ce choix dépendra du profil de l'utilisateur et de ce qu'il veut faire sur l'application à ce moment.



a) Menu conducteur

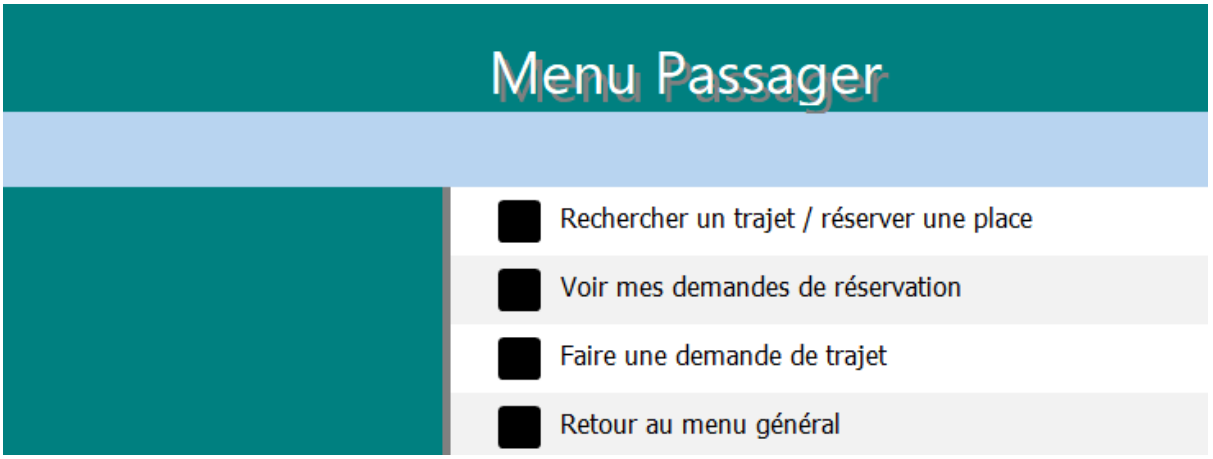
Si l'utilisateur a choisi de consulter le menu conducteur, il arrive sur l'interface suivante :



Sur ce menu, l'utilisateur se voit proposer différentes actions. Il peut par exemple ajouter une de ses voitures ou proposer un trajet. Pour faire une de ces actions, il suffit à l'utilisateur de cliquer sur le bouton correspondant comme pour l'action précédente (choix du menu). L'utilisateur peut également retourner au menu général.

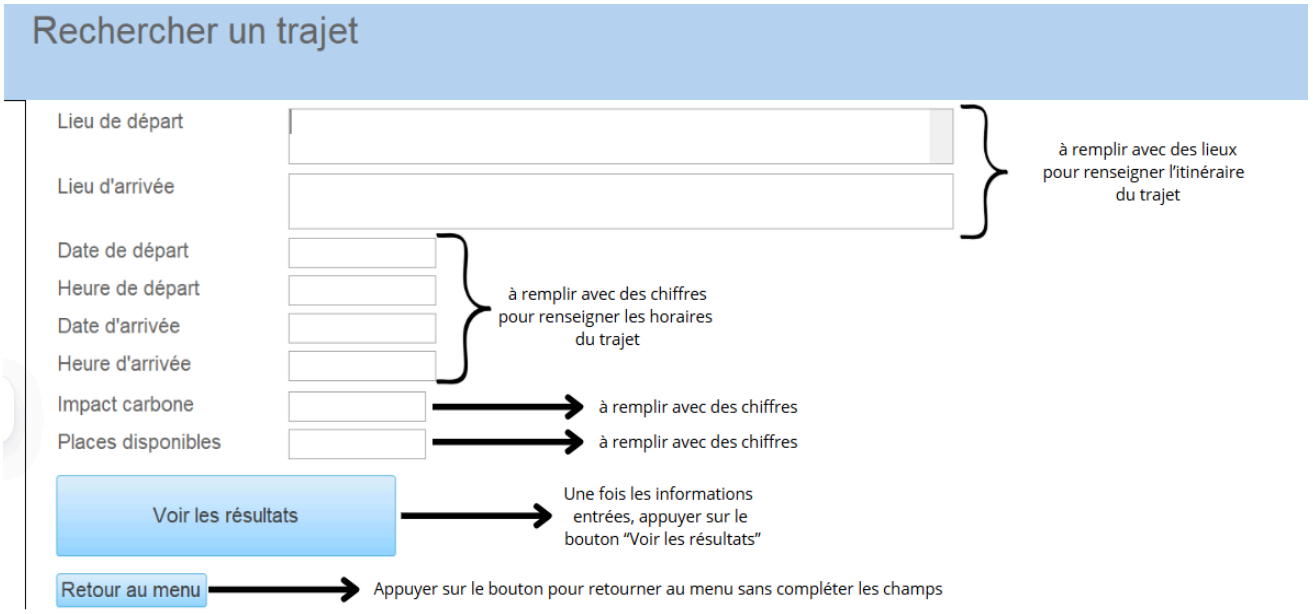
b) Menu passager

Si l'utilisateur a choisi de consulter le menu passager, il arrive sur l'interface suivante :



3) Effectuer une action

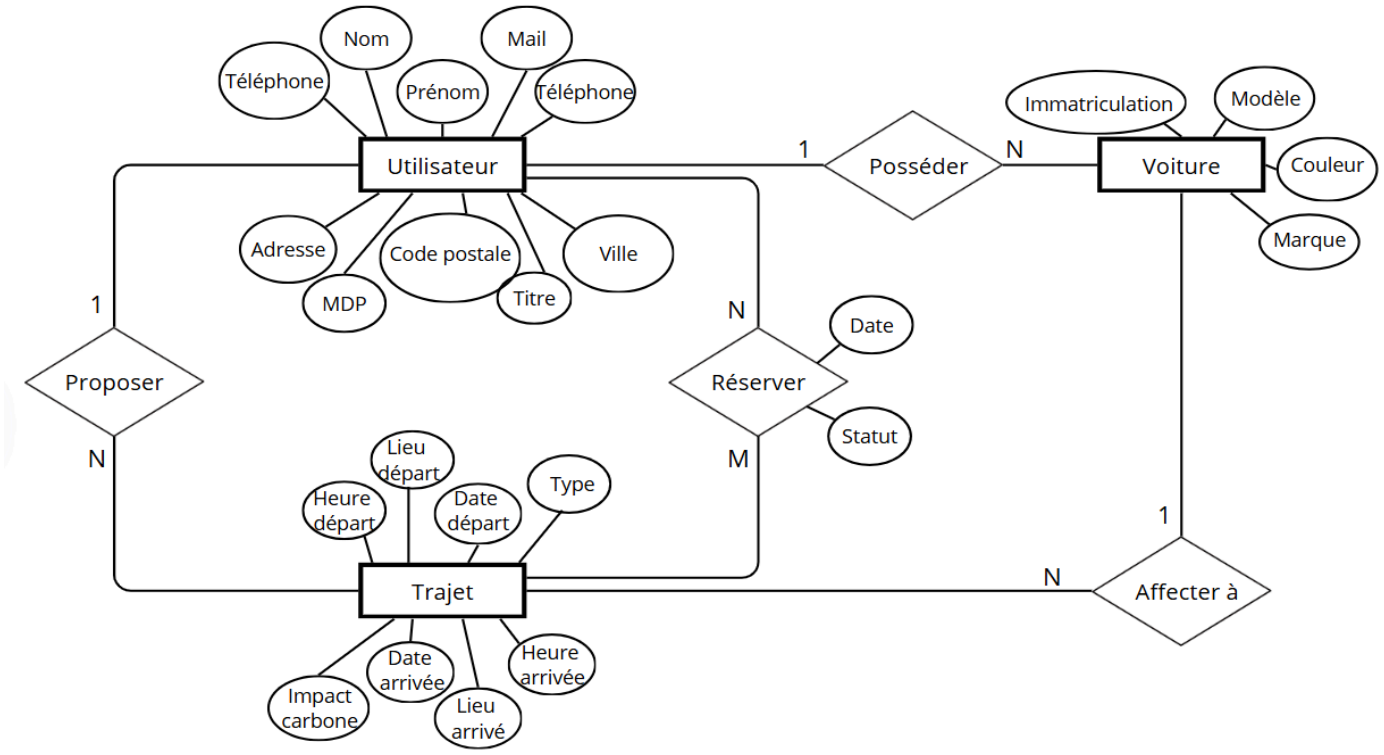
L'étape finale de l'utilisation de cette application est de faire une action, comme par exemple publier un trajet, consulter des demandes… Nous allons prendre pour exemple l'action de faire une réservation sur un trajet pour un passager. Après avoir cliqué sur le bouton correspondant, un formulaire s'ouvre. Le formulaire qui apparaît souvent possède plusieurs champs de texte que l'utilisateur doit remplir. Un bouton "retour au menu" est toujours présent au cas où l'utilisateur se trompe d'action.



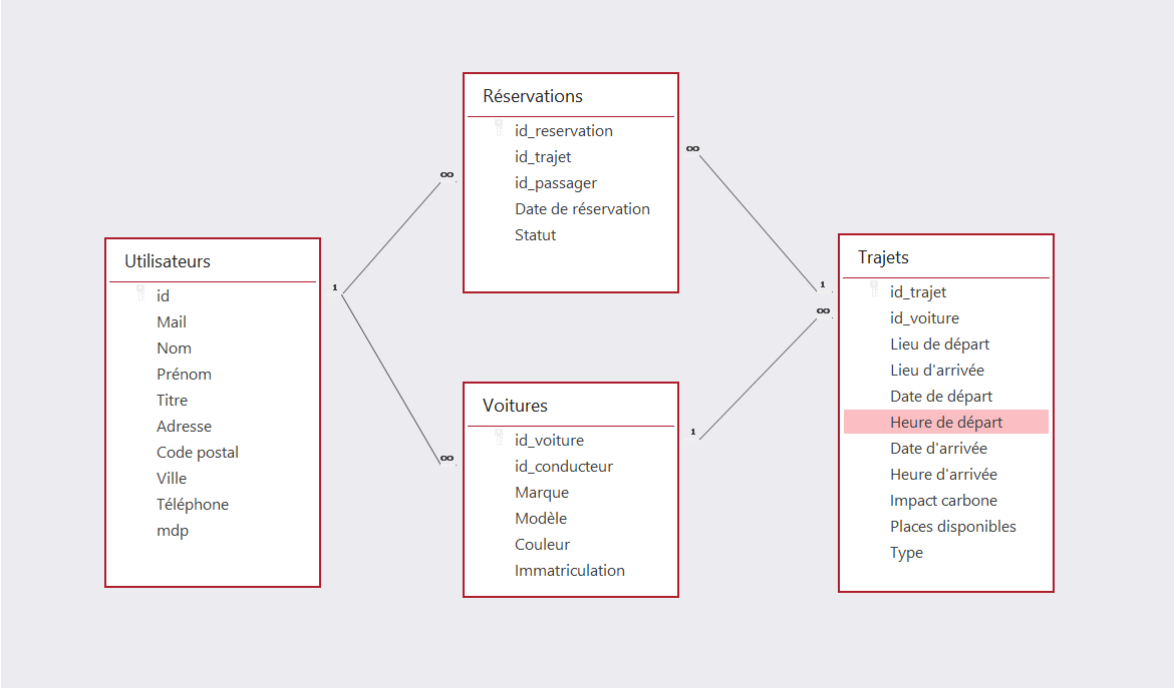
Ici, l'utilisateur peut renseigner les champs de sa recherche. Le lieu de départ et d'arrivée sont obligatoires, tandis que tous les autres champs sont facultatifs. Lors de la saisie de l'heure de départ et/ou d'arrivée, la requête de recherche prendra en compte une marge de plus ou moins 1 heure par rapport à ce qui a été indiqué.

III/ Modèle conceptuel et Modèle logique

1) Modèle conceptuel



2) Modèle logique



IV/ Fonctionnement technique

Au vu du projet, il est assez difficile de décrire en pseudo-code le fonctionnement de l'application. Nous allons plutôt présenter le fonctionnement général de nos requêtes et formulaires, car ils sont tous très similaires.

1) Requêtes

Il faut d'abord savoir que pour chaque requête, les informations des critères sont récupérées directement via la valeur de l'élément d'un formulaire, tel que [Formulaires]![Nom du formulaire]![Nom de l'élément]. Notamment, à chaque fois que l'on a besoin de l'id de l'utilisateur, on le récupère sur le formulaire de connexion, sur lequel se trouve un champ invisible ayant pour valeur l'id de l'utilisateur lorsque la connexion a réussi. Cet id est chargé automatiquement au clic sur Connexion en exécutant une macro qui récupère l'id si le mail et le mot de passe sont corrects. Nous prendrons l'exemple d'une requête de sélection, d'ajout et de modification.

Ajout : "ajouterRéservation"
Cette requête ajoute une ligne à la table Réservations. Elle est exécutée par un passager quand il effectue une demande de réservation sur un trajet existant. Pour cela, on récupère l'id du passager ainsi que l'id du trajet concerné, provenant du formulaire de recherche de trajet. On définit le statut sur "En attente", puis la demande pourra être acceptée ou refusée par le conducteur de ce trajet.

Champ :	Expr1: [Formulaires]![Demander un trajet]![id_trajet]	Expr2: [Formulaires]![Connexion]![txtIdUtilisateur]	Expr3: Date()	Expr4: "Accepté"
Table :				
Tri :				
Ajouter à :	id_trajet	id_passager	Date de réservation	Statut
Critères :				
Ou :				

Sélection : "voirListeDemandesTrajet"
Cette requête récupère la liste des demandes en attente sur un trajet choisi par le conducteur. Deux critères sont appliqués : d'abord, on cherche les réservations qui concernent un trajet en particulier. Pour cela, on récupère donc l'id du trajet depuis le formulaire de sélection de trajet par le conducteur. Puis, on ne sélectionne que les réservations dont le statut est "En attente".

Champ :	Prénom	Nom	Téléphone	Date de réservation	id_trajet	id_reservation	Statut
Table :	Utilisateurs	Utilisateurs	Utilisateurs	Réservations	Réservations	Réservations	Réservations
Tri :							
Afficher :	☑	☑	☑	☑	☑	☑	☐
Critères :					[Formulaires]![Sélection		"En attente"
Ou :							

Modification : "accepterDemandeRéservation"

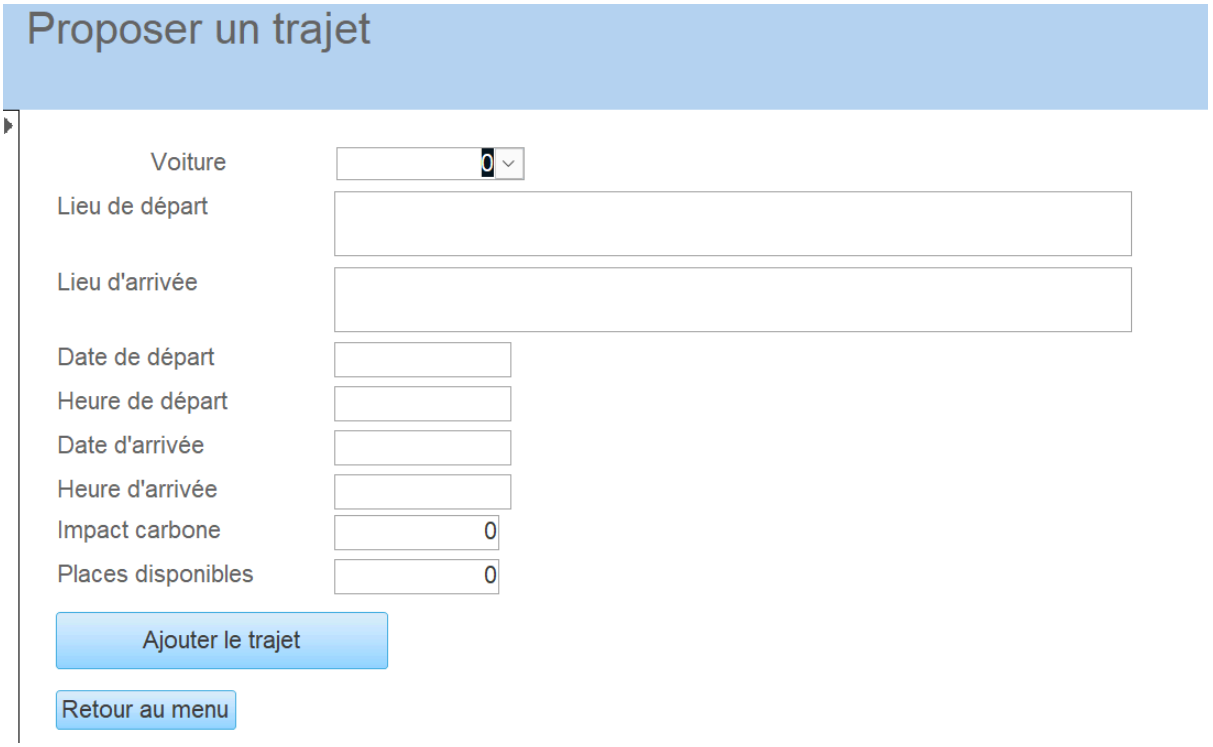
Cette requête modifie le statut d'une réservation de "En attente" vers "Accepté". Pour cela, on indique simplement la nouvelle valeur à jour, puis on utilise un critère sur l'identifiant de la réservation, récupéré sur le formulaire affichant les informations sur une réservation.

Champ :	Statut	id_reservation
Table :	Réservations	Réservations
Mise à jour :	"Accepté"	
Critères :		[Formulaires]![Demandes sur le trajet]![id_reservation]
Ou :		

2) Formulaires

Exemple 1 : Proposer un trajet

Ce formulaire permet d'inscrire un trajet dans la table correspondante avec les informations que l'on a indiqué. Toutes les informations sont obligatoires. La seule logique importante utilisée dans ce formulaire est la sélection automatique des véhicules de l'utilisateur : la liste déroulante utilise comme source la requête récupérerVéhiculesUtilisateur, qui se sert de l'id utilisateur de la page Connexion pour afficher ses voitures. Il est nécessaire d'avoir ajouté au préalable au moins 1 véhicule à son compte pour pouvoir déposer un trajet, car ces derniers sont associés au véhicule et non au conducteur.



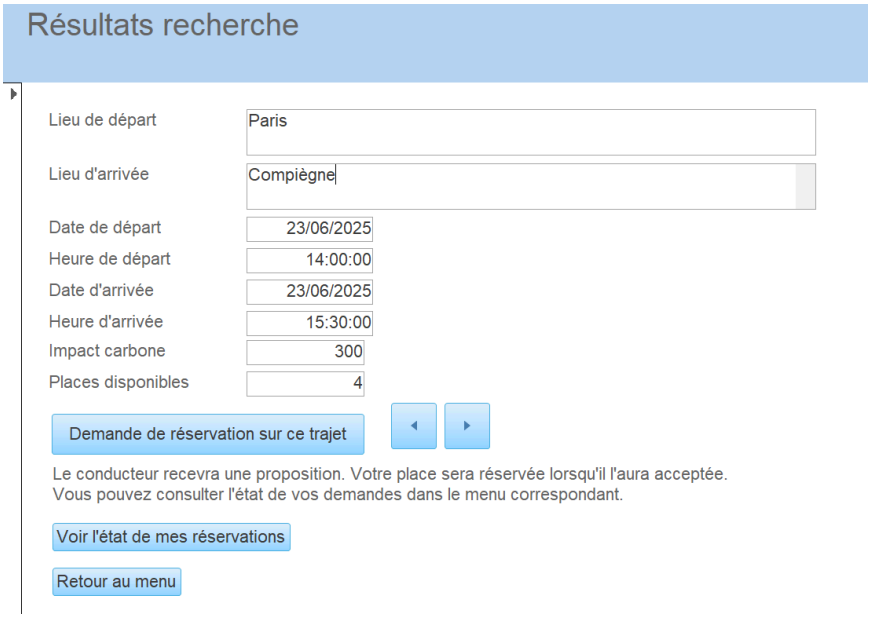
Exemple 2 : Rechercher

Ce formulaire permet de rechercher un trajet existant à partir de critères. Le strict minimum est d'indiquer le lieu de départ et d'arrivée. Les autres champs sont

facultatifs. Si l'on indique une heure de départ et/ou d'arrivée, la recherche s'effectuera sur la plage H-1 à H+1. Tout le travail est effectué grâce aux requêtes. Quasiment aucune logique ne se trouve dans les formulaires. Nous avons d'abord construit les requêtes, puis utilisé l'assistant de formulaire sur ces requêtes. Enfin, les boutons comme Voir les résultats affichent simplement le formulaire de résultat, qui exécutera la requête avant de s'afficher automatiquement.

Exemple 3 : Résultats recherche

Une fois la recherche effectuée, on obtient la page des résultats. On a la possibilité de défiler entre les différents résultats qui concordent avec notre recherche, puis on peut demander à réserver une place sur ce trajet, qui doit ensuite être confirmée par le conducteur. On peut ensuite suivre ses réservations via un état dédié.



V/ Retour bêta-testeur

Une première version de notre base de données a été testé par des utilisateurs afin d'avoir des retours constructifs dans le but d'améliorer le projet. A l'origine, deux tables différentes servaient à l'enregistrement des conducteurs et des passagers. Ce système peut sembler être le plus intuitif afin de distinguer, notamment, les utilisateurs pouvant proposer des trajets (conducteur) et ceux pouvant les réserver (passager). Le problème de ce système survient dans le cas où un utilisateur souhaite être conducteur et passager. En effet, lorsqu'il se serait connecté, il aurait été dans un menu réservé au conducteur, s'il était inscrit en tant que conducteur, ou un menu réservé au passager dans le cas contraire. Nous avons donc résolu le problème en ne créant qu'une table utilisateur. La personne pourra alors naviguer à travers tous les onglets. De plus, cela évite la redondance des informations.

Lors du bêta-test, les passagers n'avaient aucun moyen d'avoir accès à l'historique de leurs réservations. Cela est en plus très utile pour faire un point sur les trajets effectués si on souhaite faire un bilan monétaire ou environnemental de nos déplacements. Ainsi, nous avons rajouté une requête qui précède un état afin de voir tous les trajets acceptés, refusés et en cours. Le deuxième point positif de cet état est de voir l'actualisation des trajets qui sont en cours d'acceptation.

V/ Discussion

a) Contributions

Nous avons réalisé le modèle conceptuel et le modèle logique tous ensemble. Nous nous sommes répartis les différentes fonctionnalités en réalisant les requêtes et formulaires associés, puis nous avons ajouté nos différentes pages au menu à mesure de l'avancement.

i) Partie de Raphaël seul

Etant à l'aise avec le langage VBA, je me suis occupé d'ajouter un bouton permettant à l'utilisateur de voir son mot de passe lors de la connexion. J'ai aussi ajouté une condition sur la textbox "mail" qui ne peut être valide que sous la forme "texte@texte.texte" où "texte" renvoie à une suite de caractères.

ii) Partie de Simon et Raphaël

Simon et moi ayant un peu plus de difficultés dans l'élaboration des requêtes et des états, nous avons fait nos parties ensemble afin de pouvoir discuter des procédures à mettre en plus.

Pour commencer, nous avons conçu les tables et les formulaires pour que le conducteur puisse ajouter une voiture et proposer un trajet en ajoutant les masques de saisie aux champs "immatriculation", dates et heures de départ et d'arrivée. Il a aussi fallu ajouter une liste déroulante pour choisir la bonne voiture lors de l'ajout d'un trajet.

Ensuite, nous avons permis au conducteur de voir la liste de ses trajets à l'aide de la requête "listeRequeteDeTrajet" et en y associant un formulaire "Sélectionner un trajet". Dans ce formulaire, le conducteur peut choisir un de ses trajets et y voir les demandes de réservations et les réservations déjà faites. Le bouton permettant de voir la demande de réservation est associé à un formulaire "Demandes sur la trajet" relié à la requête "RecupererListeDemandes". On peut alors accepter ou refuser la demande du passager. Ceci est fait à l'aide des requêtes "accepterdemandepassager" ou "refuserdemandepassager" qui modofie le champ "statut" de la table réservation. On peut aussi voir, à l'aide de l'état "Listes des passsagers sur le trajet" associé à la requête "recupererListePassager" de voir tous les passagers confirmés du voyage en question.

Enfin, nous avons développé l'option pour les passagers de proposer un trajet et donc, pour les conducteurs, de pouvoir les vérifier et les accepter ou non. Le passager ouvre le formulaire "Demander un trajet". Il ajoute un trajet dans la table de type "demande". Le conducteur ouvrira lui le formulaire "Liste des requêtes de trajet". Avec le bouton "prendre en charge", la requête "AccepterRequêteDeTrajet" modifie la table trajet en lui associant la voiture, le nombre de places dispo ainsi que le nouveau type de trajet qui devient "proposition".

iii) Partie de Youssef

Je me suis occupé de réaliser toute la partie sur la recherche de trajet et la réservation de places. j'ai commencé par créer les différentes requêtes qui allaient me servir : ajouter une réservation, récupérer la liste des demandes de réservation sur un trajet, rechercher un trajet par critères, voir l'état de ses demandes de réservation… Une fois mes requêtes prêtes, j'ai pu générer les formulaires associés. J'ai ensuite ajouté les champs nécessaires provenant éventuellement d'autres tables, j'ai ajouté les boutons de navigation et d'action. J'ai pu ensuite faire un état basé sur une requête pour récupérer toutes les réservations au nom de l'utilisateur.

Au niveau général, j'ai ajouté toutes les macros nécessaires aux autres formulaires, notamment pour les événements clic et chargement. J'en ai profité pour copier le bouton permettant de fermer la fenêtre sur les autres formulaires.

Enfin, je me suis chargé de réaliser la macro sur la page Connexion. Cette dernière exécute une requête pour vérifier le mail et le mot de passe, et si une entrée est trouvée avec les deux, récupérer l'identifiant de l'utilisateur associé et remplacer un champ invisible par cette valeur. Ainsi, il est possible d'utiliser partout ailleurs l'id de l'utilisateur connecté (tant que le formulaire de Connexion reste ouvert).

b) Voies d'améliorations

Il existe diverses voies d'améliorations pour rendre l'application meilleure. Tout d'abord, l'interface peut être améliorée car elle est basique pour le moment, bien que ce soit fonctionnel et simple d'utilisation. De nombreuses autres fonctionnalités et vérifications pourraient être ajoutées, notamment :

- Une meilleure gestion des contraintes, par exemple le fait qu'un conducteur ne puisse pas réserver une place sur son trajet, ou trouver un moyen de se passer de l'utilisateur virtuel qui est utilisé lorsqu'un passager demande un trajet, ou bien mieux vérifier les informations saisies dans les formulaires, par exemple lorsqu'un conducteur accepte une demande de trajet ;
- Rendre les états plus lisibles ;
- Ajouter quelques questions et algorithmes permettant d'établir l'empreinte carbone, car l'utilisation actuelle n'est pas pratique ni parlante ;
- Mettre en place la possibilité de définir un prix sur le trajet, ou bien un calcul automatique selon la distance entre le départ et l'arrivée ;
- Ajouter plus de contraintes sur la création du mot de passe (longueur, complexité) ;
- Choisir la tolérance sur l'heure dans la recherche de trajet ;
- Vérifier le nombre de places disponibles et soustraire le nombre de réservations acceptées pour ne pas saturer les véhicules.